

Topic - CSS Basics
Mentor - Sushant Goyal
Date - 23 August 2025

CSS Basics

1. What is CSS
2. ID and class
3. Pseudo classes
4. Display Property

HTML - HyperText Markup Language

Hypertext - it is linking the pages or we can jump from one page to another

.html - file extension

`<!DOCTYPE html>` - to specify the version of the HTML used

`<HTML></HTML>` - wraps the complete structure

`<Head></head>` - meta information and css or js linking

`<body></body>` - it is the main area for the page structure

`<forms></forms>` - to take inputs from the user

`<input type="<TYPES>" />` - to specify the type of the input field
- text, email, number, password, date, submit

action in forms - the callable or the link it will redirect to once the form is submitted

method - to define GET or POST submission

label - to define the field by name

name - required for the data to be present in the submitted data to URL

id - required for label to be tagged to the field

required - it marks the field to be essentially filled before submitting the form

`<table></table>` - structure the html data

`<thead></thead>` - for the table header

`<tbody></tbody>` - for the table body

`<tr></tr>` - table row irrespective of thead or tbody

`<th></th>` - to mark the cell as the header

`<td></td>` - to add the data into cell within the body

Semantic tags - easily understandable and SEO ready tags

SEO - Search Engine Optimization

Why SEO is important - to improve the page ranking in the search results

Main - for main content

article - for blogs or articles

nav - to mark the navigation bar or we say the menu items

section - to define a section within the page

footer - to define the footer links or the copyright area

header - to define the intro or nav area

strong - to mark the text as bold

em - to mark the text in italics.

CSS - Cascading Style Sheet

It controls the visual presentation of the HTML content

Role of CSS:

1. it control the visual presentation
2. the fonts - style, size, weight
3. colors, spacing
4. page layout - using grid and flexbox
5. animations and interaction (pseudo classes)

file extension - .css for css specific files

CSS inclusion in HTML

1. inline css
2. internal css
3. external css

Inline CSS - when we write the style within the html element

```
<h1 style="color:red;">Heading 1</h1>
```

whenever we are providing any property in the css it has to end with a semicolon (;)

Internal CSS - when we write the style in the same html file

```
<head><style></style></head>
```

We can add the style tag within the body element BUT IT IS NOT RECOMMENDED BY W3C (WORLD WIDE WEB CONSORTIUM) - who defines the html rules

External CSS - to include the CSS from an external source using the link tag

link - it creates a link between the external document with the current document

Properties of link:

rel - to define the relationship of the file being imported.

rel="stylesheet"

href - to define the source of the file to be imported

Purpose of using external CSS

1. separates the style from the html document
2. it promotes the reuse of the same style
3. code becomes more readable

CSS - Cascading Style Sheet

Cascading means styles can come from multiple sources and how it is prioritized

1. Location - Inline > Internal > External
2. Specificity - ID > class > element
3. Order of appearance - Later overrides earlier

Selectors in CSS

Selectors are reference that target HTML elements so we can apply styles they tell CSS: Here it is now add the styles to this part of the HTML

1. Element selectors

targets the elements by the tag name.

```
h1 {  
  color: red;  
}  
// it will apply this style to all the <h1> tags
```

2. Class Selectors

targets the elements with the same class.

to define a class selector, we use dot `.`

to add class to html tag we use class attribute.

Reusable: same class can be applied to multiple elements

```
.heading-1 {  
  color: blue;  
}  
// it will apply the style to the element with class as heading-1
```

3. ID Selector

targets the element with the given ID

to define an ID selector we use `#`

to add ID to html tag we use the id attribute.

**** Meant to be unique - used only once per page.**

```
#one {  
  color: blue;  
}
```

NOTE: Class and ID selectors should begin with a character.

```
#heading-1 {  
  color: red  
}
```

```
.heading-1 {  
  color: blue  
}
```

while importing from an external file, ID takes more priority because of specificity rule

border - to apply the border - the width, the style of border, color of border

width - to manage the size it consumes for the width of screen

height - to control the height of the element

font-size - to control the size of the font

font-weight - add weightage to the text

font-style - to change the font style - italic, oblique or normal

font-family - to change the font typeface - SANS, OpenSans, Ariel

font-variant - control small-caps

color - to change the color of the text

background color - to change the background

margin - adding spaces around the element

padding - adding the space within the element

'cursor' - defines how the mouse icon will be displayed based on the element

Defining the sizes:

1. px (pixels) - to define the sizes into pixels
2. pt (points) - used for print
3. em - Relative to the parent element size
font-size of parent as 10px, font-size: 1.5em - font size will be 15px
4. rem - Relative to the root element (<html>)
1 rem = 16 pixels
5. % - Relative to the parent elements size.

Display property

It determines how an element behaves in the layout flow of the page

1. block - full-width, starts on the new line.
2. inline - shares line with other content. width or height can not be set
3. none - removes element from the layout
4. inline-block - combines behaviour of both inline and block. element behaves like inline element but we can still set the height or width.

Pseudo Classes:

these allow us to style the element in a specific state or situation

we need not to add any JS code to interact, using css only will react to the states.

:<state> - defining the pseudo classes or states

✓ ✓
anchor tag

:link - color blue - the default display state

:visited - when the link has been visited

:hover - while the mouse is on the anchor tag

:active - while the anchor tag is being clicked

button tag

:hover

:active

Specificity Rules

1. Location - Inline > Internal > External

2. Specificity - ID > class > element

3. Order of appearance - Later overrides earlier

!important

special flag that can add a higher specificity to a CSS declaration

It ensures that a particular rule is applied regardless of location or specificity of other rules

When this is to be used:

1. when we are using third party CSS libraries

to negate the effect of imported styles

2. when we have to override the inline styles
3. There is a specific rule that we want to override
eg- i want to update the font size for one of the h1 elements

Downsides of !important

1. it can lead to specificity wars: to overcome multiple instances of !important, we have to use !important more extensively.
2. Makes debugging harder: using multiple !important will make things tough to identify which place the rule is exactly being updated from.
3. Less maintainable - overuse of !important can make stylesheets harder to maintain.

Best Practices to use !important:

1. Limiting its use - when absolutely important
2. Being specific - writing more specific selectors

to add style for multiple elements we can use `,` to separate
to be more specific we can join the different selectors
eg h1#heading-1

Task:

Add styling to the portfolio created in the previous session